The architecture consists of two primary modules:

- A **vision node**, responsible for detecting and tracking finger motion using a deep learning model.
- A **control node**, which interprets the tracked motion, constructs a path, and generates velocity commands using a path-following algorithm.

The objective of the system is to achieve smooth, intuitive, and stable robot motion based on user gestures.

System Architecture

The system follows a modular ROS2-based design:

- The **vision node** captures camera input, performs object detection using YOLO, and publishes:
  - Finger position (/finger_pos)
  - Motion state (/finger_moving)

- The **control node** subscribes to:
  - Finger position
  - Motion state
  - Robot odometry

It processes this data to:

- Generate a path from finger motion
- Apply a control algorithm
- Publish velocity commands (/cmd_vel)

This separation ensures scalability and real-time performance.

Vision-Based Finger Tracking

Finger Detection

Finger detection is performed using a trained YOLO (You Only Look Once) model. YOLO processes each frame and outputs bounding boxes along with confidence

scores. Among multiple detections, the bounding box with the highest confidence is selected to represent the finger.

The center of the bounding box is computed as:

$$x_c = \frac{x_1 + x_2}{2}, y_c = \frac{y_1 + y_2}{2}$$

This center point is used as the finger position.

Noise Reduction Using Smoothing

Raw detections from the model may exhibit jitter due to small variations between frames. To address this, an Exponential Moving Average (EMA) filter is applied:

$$x_s = \alpha x_{raw} + (1 - \alpha) x_{prev}$$
$$y_s = \alpha y_{raw} + (1 - \alpha) y_{prev}$$

where $\alpha$is a smoothing factor. This ensures stable tracking and prevents erratic robot motion.

Movement Detection

To determine whether the finger is actively moving, the system computes the Euclidean distance between consecutive positions:

$$d = \sqrt{(x - x_{prev})^2 + (y - y_{prev})^2}$$

If this distance exceeds a predefined threshold, the finger is considered to be moving. A time-based filter is further applied: if no movement is detected within a specified duration, the system classifies the finger as stationary.

This prevents false detections caused by minor noise.

Output Generation

The vision node publishes:

- A continuous stream of finger positions
- A Boolean flag indicating motion state

These outputs form the input for the control system.

Path Generation from Finger Motion

Coordinate Transformation

Finger positions are initially in image coordinates. These are mapped to the robot's world frame using a scaling transformation:

$$x_w = x_r + (p_x - p_{ref}) \cdot S$$
$$y_w = y_r - (p_y - p_{ref}) \cdot S$$

where:

- S is a scaling factor
- $(p_{ref})$is the reference finger position
- $(x_r, y_r)$is the robot's current position

This mapping allows the finger trajectory to be interpreted as a path in the robot's environment.

Path Construction

As the user moves their finger, points are appended to a path. To avoid redundancy, new points are only added if they are sufficiently distant from the previous point.

This ensures:

- Efficient path representation
- Reduced computational load

State Management

The control node operates using a state machine:

- **IDLE**: No input detected
- **DRAWING**: Recording finger motion
- **COMPLETING**: Robot follows the generated path

Transitions between states depend on whether the finger is moving or stationary.

5. Robot Motion Control Using Pure Pursuit

Lookahead Point Selection

The Pure Pursuit algorithm is used for path tracking. A lookahead point is selected such that it lies at a fixed distance ahead of the robot on the path.

This is computed by finding the intersection of:

- A circle centered at the robot (radius = lookahead distance)
- The path segments

Coordinate Transformation

The lookahead point is transformed into the robot's local coordinate frame:

$$x_{local} = \cos\ \theta \cdot dx + \sin\ \theta \cdot dy$$
$$y_{local} = -\sin\ \theta \cdot dx + \cos\ \theta \cdot dy$$

This simplifies control calculations.

Curvature Calculation

The curvature required to reach the lookahead point is:

$$\kappa = \frac{2 \cdot y_{local}}{d^2}$$

This determines how sharply the robot must turn.

Velocity Computation

Linear velocity is adjusted based on curvature:

- High curvature → lower speed
- Low curvature → higher speed

Angular velocity is computed as:

$$\omega = v \cdot \kappa$$

To ensure safety, angular velocity is clamped within predefined limits.

Adaptive Lookahead

The lookahead distance is dynamically adjusted:

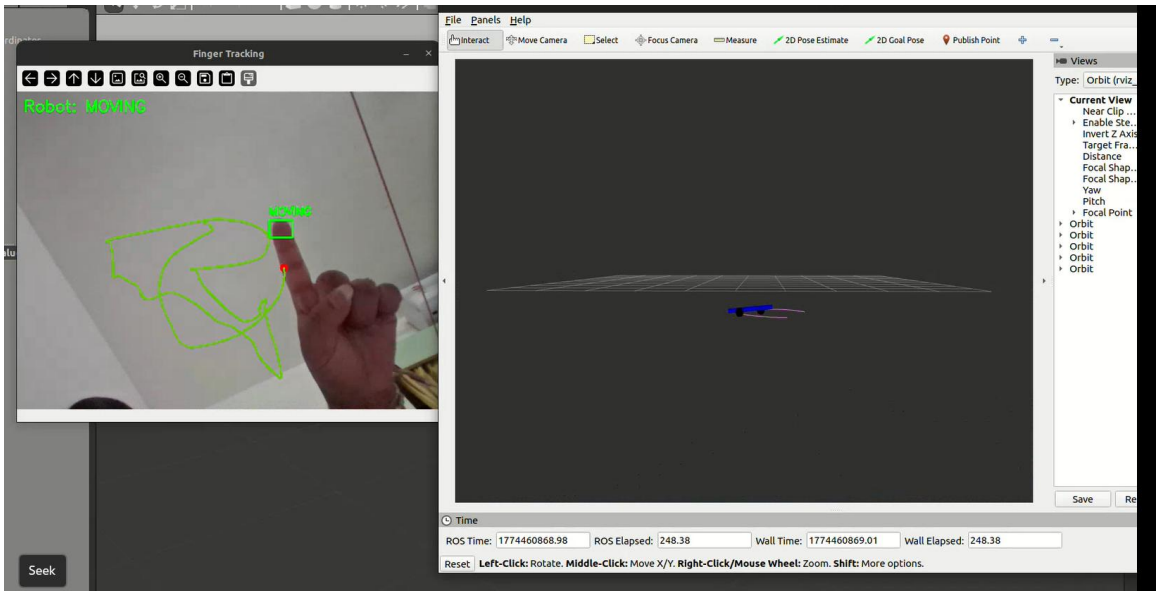
- Larger for straight paths
- Smaller for sharp turns

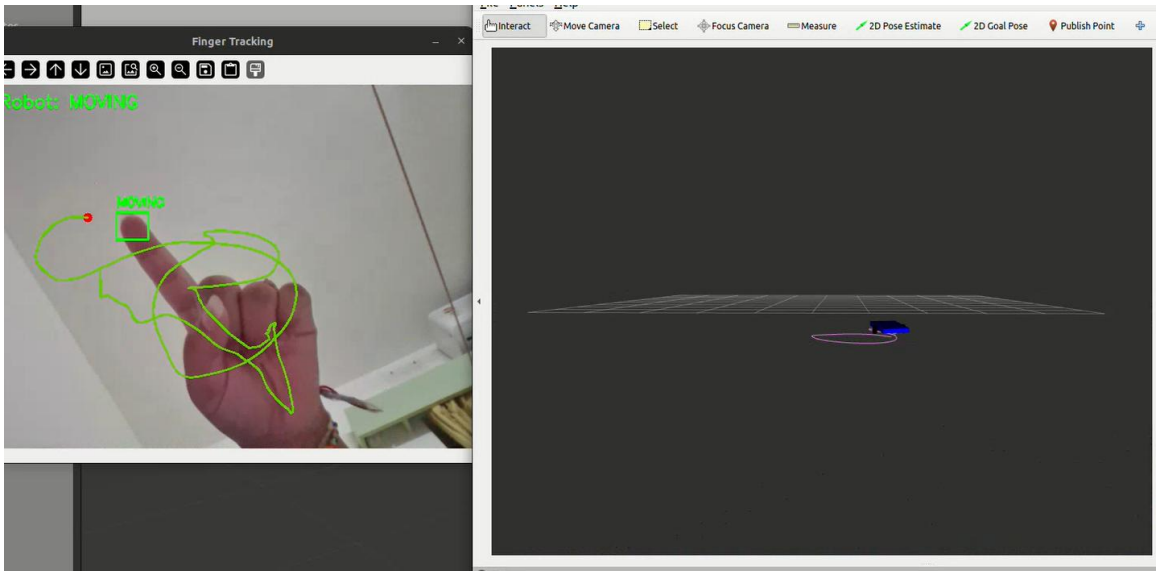
This improves both stability and responsiveness.

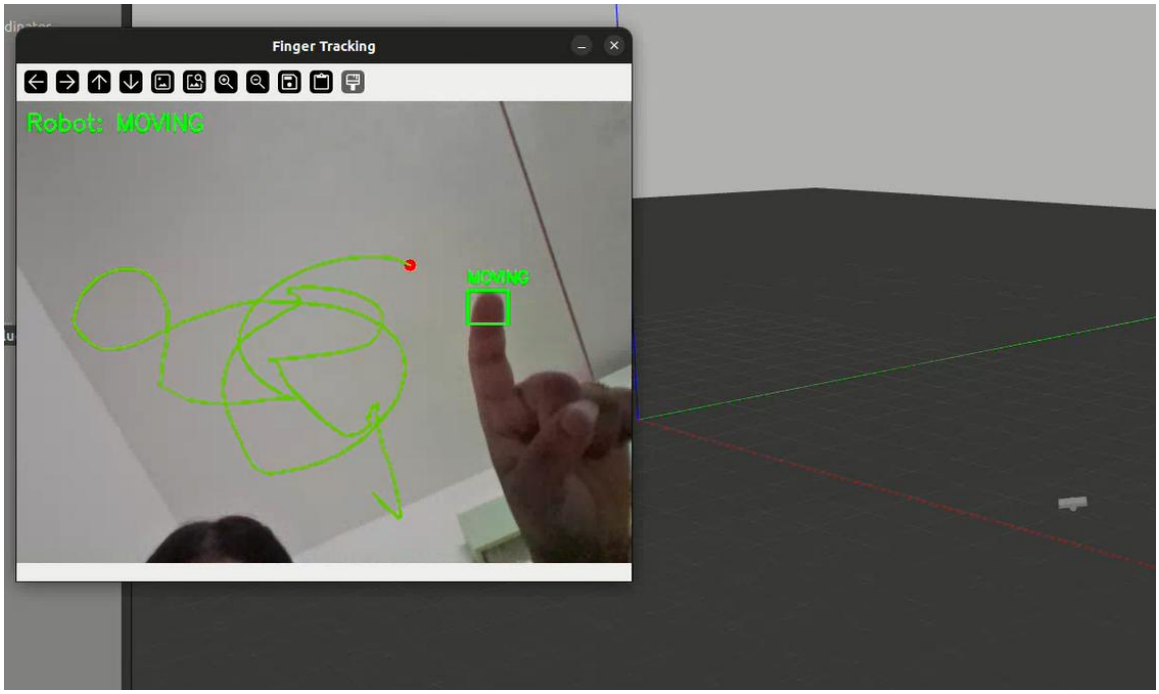
6. Stopping Criteria

The robot stops under the following conditions:

- The final path point is reached within a tolerance
- The finger stops moving
- The path contains insufficient points


Finger Tracking
Robot: MOVING
MOVING
File Panels Help
Interact
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
2D Goal Pose
Publish Point
Views
Type: Orbit (rviz
Current View
Near Clip ...
Enable Ste...
Invert Z Axis
Target Fra...
Distance
Focal Shap...
Focal Shap...
Yaw
Pitch
Focal Point
Orbit
Orbit
Orbit
Orbit
Orbit
Save
Time
ROS Time: 1774460868.98
ROS Elapsed: 248.38
Wall Time: 1774460869.01
Wall Elapsed: 248.38
Reset
Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel: Zoom. Shift: More options.
Seek


Finger Tracking
Robot: MOVING
MOVING
Interact
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
2D Goal Pose
Publish Point


Finger Tracking
Robot: MOVING
MOVING